



# **IT 252 DATABASE SYSTEMS**

**More SQL: Complex Queries,  
Triggers, Views, and  
Schema Modification**

**Dr. Jayashree T R**

# Course Outline

## Course Plan: Theory:

### Part A: Parallel Computer Architectures

Week 1: Introduction to the course, highlighting the data and databases, and related basic concepts. Advantages and need for a database system.

Week 2: Attributes, tuples, relational schema, conceptual model, introduction to SQL-DML, DDL, creating relations/ tables, access and manipulate data.

Week 3,4: Data model, relational model, and SQL, relational data model and relational database constraints, SQL data definition and data types, specifying constraints, basic retrieval queries, complex queries, triggers, views, schema modification.

Week 5-6: ER model development, entity types/sets, attribute, relationship types/sets, simple employee database conceptual design using ER concepts, ER to relational mapping algorithm, EER model.

# Course Outline

- Week 7,8 : Functional dependency definition, need of normalization, anomalies and redundant information in tuples, normalization steps (1NF, 2NF, 3NF), BCNF, introduction to the higher normal forms.
- Week 9: Procedural query languages: relational algebra and SQL-SELECT, PROJECT, RENAME, binary and JOIN operations with examples.
- Non-Procedural query languages: relational calculus, and SQL-tuple relational calculus, existential and universal quantifier with examples.
- Week 10,11 : Transaction management, schedule and serializability, concurrency control, 2-phase lock, recovery mechanism: undo/redo values.
- Week 12,13: Disk storage- basic file structures, ordered/unordered. binary search, hashing, indexing, importance of indexes, primary and secondary indexing methods, clustered, B tree, B++ trees.
- Week 14: Current trends in database system, introduction to data warehousing, data mining.
- Practical: MySQL commands
- Project: Team of 2 or 3 members

# More Complex SQL Retrieval Queries

- Additional features allow users to specify more complex retrievals from database: such as Nested queries, joined tables, and outer joins (in the FROM clause), aggregate functions, and grouping.

# Comparisons Involving NULL and Three-Valued Logic

- Meanings of NULL
  - **Unknown value**
  - **Unavailable or withheld value**
  - **Not applicable attribute**
- Each individual NULL value considered to be different from every other NULL value
- SQL uses a three-valued logic:
  - TRUE, FALSE, and UNKNOWN (like Maybe)
- **NULL = NULL comparison is avoided**

## meanings of NULL:

1. **Unknown value.** A person's date of birth is not known, so it is represented by NULL in the database. An example of the other case of unknown would be NULL for a person's home phone because it is not known whether or not the person has a home phone.
2. **Unavailable or withheld value.** A person has a home phone but does not want it to be listed, so it is withheld and represented as NULL in the database.
3. **Not applicable attribute.** An attribute LastCollegeDegree would be NULL for a person who has no college degrees because it does not apply to that person.

## Comparisons Involving NULL

- SQL allows queries that check whether an attribute value is NULL. Rather than using = or <> to compare an attribute value to NULL, SQL uses the comparison operators **IS** or **IS NOT**.
- This is because SQL considers each NULL value as being ***distinct*** from every other NULL value, so equality comparison is not appropriate.

Query 18. Retrieve the names of all employees who do not have supervisors.

```
Q18:  SELECT  Fname, Lname
      FROM    EMPLOYEE
      WHERE   Super_ssn IS NULL;
```

# Aggregate Functions in SQL

- Used to summarize information from multiple tuples into a single-tuple summary
- Built-in aggregate functions
  - **COUNT**, **SUM**, **MAX**, **MIN**, and **AVG**
- **Grouping**
  - Create subgroups of tuples before summarizing
- To select entire groups, **HAVING** clause is used
- Aggregate functions can be used in the **SELECT** clause or in a **HAVING** clause

The **COUNT** function returns the number of tuples or values as specified in a query.



# Aggregate Functions in SQL

Query 19. Find the sum of the salaries of all employees, the maximum salary, the minimum salary, and the average salary

## Renaming Results of Aggregation

- Following query returns a single row of computed values from EMPLOYEE table:

```
Q19:      SELECT  SUM (Salary), MAX (Salary), MIN (Salary), AVG  
           (Salary)  
           FROM    EMPLOYEE;
```

- The result can be presented with new names:

```
Q19A:     SELECT  SUM (Salary) AS Total_Sal, MAX (Salary) AS  
                 Highest_Sal, MIN (Salary) AS Lowest_Sal, AVG  
                 (Salary) AS Average_Sal  
           FROM    EMPLOYEE;
```

# Aggregate Functions in SQL (cont'd.)

- NULL values are discarded when aggregate functions are applied to a particular column

**Query 20.** Find the sum of the salaries of all employees of the 'Research' department, as well as the maximum salary, the minimum salary, and the average salary in this department.

```
Q20:  SELECT    SUM (Salary), MAX (Salary), MIN (Salary), AVG (Salary)
      FROM      (EMPLOYEE JOIN DEPARTMENT ON Dno=Dnumber)
      WHERE     Dname='Research';
```

The **JOIN** operator is used to combine rows from EMPLOYEE table with the DEPARTMENT table.

**Queries 21 and 22.** Retrieve the total number of employees in the company (Q21) and the number of employees in the 'Research' department (Q22).

```
Q21:  SELECT    COUNT (*)
      FROM      EMPLOYEE;
```

```
Q22:  SELECT    COUNT (*)
      FROM      EMPLOYEE, DEPARTMENT
      WHERE     DNO=DNUMBER AND DNAME='Research';
```

# Aggregate Functions in SQL

## Working of Query 20:

- EMPLOYEE Table:

| EmpID | Name    | Dno | Salary |
|-------|---------|-----|--------|
| 1     | Alice   | 10  | 5000   |
| 2     | Bob     | 20  | 7000   |
| 3     | Charlie | 10  | 6000   |

- DEPARTMENT Table:

| Dnumber | Dname     |
|---------|-----------|
| 10      | Research  |
| 20      | Marketing |

When JOIN condition is applied (EMPLOYEE.Dno = DEPARTMENT.Dnumber), only employees with Dno = 10 will be included because the WHERE clause further filters these to just employees in the "Research" department.

Hence, the aggregation functions (SUM, MAX, MIN,AVG) would be applied to just Alice and Charlie, whose Dno is 10 (**Research** department), calculated as follows.

$$\text{SUM(Salary)} = 5000 + 6000 = 11000$$

$$\text{MAX(Salary)} = 6000$$

$$\text{MIN(Salary)} = 5000$$

$$\text{AVG(Salary)} = (5000 + 6000) / 2 = 5500$$

# Grouping: The GROUP BY Clause

- **Partition** relation into subsets of tuples
  - Based on **grouping attribute(s)**
  - Apply function to each such group independently
- **GROUP BY** clause
  - Specifies grouping attributes
- **COUNT (\*)** counts the number of rows in the group

# Examples of GROUP BY

- The grouping attribute must appear in the SELECT clause:

Q24:            **SELECT**            Dno, **COUNT** (\*), **AVG** (Salary)  
                 **FROM**            EMPLOYEE  
                 **GROUP BY**       Dno;

- If the grouping attribute has NULL as a possible value, then a separate group is created for the null value (e.g., null Dno in the above query)
- GROUP BY may be applied to the result of a JOIN:

Q25:            **SELECT**            Pnumber, Pname, **COUNT** (\*)  
                 **FROM**            PROJECT, WORKS\_ON  
                 **WHERE**            Pnumber=Pno  
                 **GROUP BY**       Pnumber, Pname;

- In **Q24**, the EMPLOYEE tuples are partitioned into groups. Each group contains the employees who work in the same department.
- The COUNT and AVG functions are applied to each such group of tuples. Notice that the SELECT clause includes only the grouping attribute and the aggregate functions to be applied on each group of tuples.

# Grouping: The GROUP BY and HAVING Clauses (cont'd.)

- **HAVING** clause

- Provides a condition to select or reject an entire group:

- **Query 26.** For each project *on which more than two employees work*, retrieve the project number, the project name, and the number of employees who work on the project.

|      |                 |                                  |
|------|-----------------|----------------------------------|
| Q26: | <b>SELECT</b>   | Pnumber, Pname, <b>COUNT</b> (*) |
|      | <b>FROM</b>     | PROJECT, WORKS_ON                |
|      | <b>WHERE</b>    | Pnumber=Pno                      |
|      | <b>GROUP BY</b> | Pnumber, Pname                   |
|      | <b>HAVING</b>   | <b>COUNT</b> (*) > 2;            |



# Combining the WHERE and the HAVING Clause

- Consider the query: we want to count the *total* number of employees whose salaries exceed \$40,000 in each department, but only for departments where more than five employees work.

- **INCORRECT QUERY:**

|                 |                       |
|-----------------|-----------------------|
| <b>SELECT</b>   | Dno, <b>COUNT</b> (*) |
| <b>FROM</b>     | EMPLOYEE              |
| <b>WHERE</b>    | Salary > 40000        |
| <b>GROUP BY</b> | Dno                   |
| <b>HAVING</b>   | <b>COUNT</b> (*) > 5; |

How? Analyze



## Practice Queries:

### Query 1:

Consider the relational table given below and assess about the following SQL queries.

Employee (Empno, Name, Department, Salary). (3)

- (i) List all the employees whose name starts with the letter 'L'. (2)
- (ii) Find the maximum salary given to employees in each department. (2)
- (iii) Find the number of employees working in 'accounts' department. (2)
- (iv) Find the second maximum salary from the table. (2)
- (v) Find the employee who is getting the minimum Salary. (2)

### Query 2:

- a) Write an SQL query for the following :
  - a) To create a table of Hospital database with minimum 5 fields
  - b) To insert two records
  - c) To add new field
  - d) To display all records.

### Query 3: Employee database. Primary keys are underlined

employee (employee-name, street, city)  
works (employee-name, company-name, salary)  
company (company-name, city)  
manages (employee-name, manager-name)

Give an expression in SQL for each of the following queries.

- Find the names of all employees who work for First Bank Corporation.
- Find the names and cities of residence of all employees who work for First Bank Corporation.
- Find the names, street addresses, and cities of residence of all employees who work for First Bank Corporation and earn more than \$10,000.
- Find all employees in the database who live in the same cities as the companies for which they work.
- Find all employees in the database who live in the same cities and on the same streets as do their managers.
- Find all employees in the database who do not work for First Bank Corporation.

## Query 4:

Consider the following relation schema

Works(Pname,Cname,salary)

Lives(Pname,Street,City)

located\_in (Cname, city)

Manager(Pname,Mgrname)

Write the SQL queries for the following

- i) Find the names of all persons who live in the city Bangalore.
- ii) Retrieve the names of all person of "Infosys" whose salary is between Rs .50000
- iii) Find the names of all persons who lives and work in the same city
- iv) List the names of the people who work for “Tech M” along with the cities they live in.
- v) Find the average salary of “Infosys” persons

## Query 5:

Consider the following relations for a database that keeps track of automobile sales in a car dealership:

CAR(Serial\_no, Model, Manufacturer, Price).

OPTION(Serial\_no, Option\_name, Price).

SALE(Salesperson\_id, Serial\_no, Date, Sale\_price)

SALESPERSON(Salesperson\_id, Name, Phone).

OPTION refers to some optional equipment installed on an automobile.

First specify the foreign keys for this schema, stating any assumptions you made. Next, populate the relation with a few sample tuples, and then give an example of an insertion in the SALE and SALESPERSON relation that violates the referential integrity constraints and of another insertion that does not.

## Query 6:

**Q.4. A company database needs to store information about employees (identified by ssn, with salary and phone as attributes), departments (identified by dno, with dname and budget as attributes), and children of employees (with name and age as attributes). Employees work in departments; each department is managed by an employee; a child must be identified uniquely by name when the parent (who is an employee; assume that only one parent works for the company) is known. We are not interested in information about a child once the parent leaves the company. Draw an ER diagram that captures this information.**

## Query 7:

Consider the following relations for a company database Application:

**Employee (Eno, Name, Sex, DOB, Doj, Designation, Basic\_Pay, Dept\_No)**

**Department (DeptNo, Name)**

**Project (ProjNo, Name, Dept\_No)**

**Works for (Eno,ProjNo,Date, Hours)**

The attributes specified for each relation is self-explanatory. However, the business rules are stated as follows. A department can control a project. An employee can work on any number of projects on a day.

However, an employee cannot work more than once on a project he/she worked on that day. The primary key are underlined.

- (i) Identify the foreign keys, Develop DDL to implement the above schema.
- (ii) Develop an SQL query to list the department number and the number of Employee in each department.
- (iii) Develop a View that will keep track of the department number, the number of employees in the department, and the total basic pay expenditure for each department.
- (iv) Develop an SQL query to list the details of employees who have worked in more than three projects on a day.

## Query 8: Painting Hire Business

### System Description:

A local businesswoman has decided to start her own Internet business, called Masterpieces Ltd, hiring paintings to private individuals and commercial companies. Because of your reputation as a database designer, she has called upon your services to design and implement a database to support her new business. At the initial planning meeting, to discuss the design, the **following user requirements** were requested.

The system must be able to manage the details of customers, paintings and those paintings currently on hire to customers. Customers are categorized as B (bronze), S (silver), G (gold) or P (platinum). These categories entitle a customer to a discount of 0%, 5%, 10% or 15% respectively.

Customers often request paintings by a particular artist or theme (eg animal, landscape, seascape, naval, still-life, etc). Over time a customer may hire the same painting more than once. Each painting is allocated a customer monthly rental price defined by the owner. The owner of the painting is then paid 10% of that customer rental price. Any paintings that are not hired within six months are returned to the owner. However, after three months, an owner may resubmit a returned painting. Each painting can only have one artist associated with it. Several reports are required from the system. Three main ones are:

1. For each customer, a report showing an overview of all the paintings they have hired or are currently hiring.
2. For each artist, a report of all paintings submitted for hire .
3. For each artist, a returns report for those paintings not hired over the past six months.

Remember to identify **key attributes** and **any foreign key** attributes.

## Query 9:

Consider the following relational database:

Customer(cno, cname, city)

Quotation(qno, qdate, description, amt\_quoted, cno)

Customer and Quotation are related with one-to-many relationship. Write MySQL queries for the following:

- i. Display customer names having quotation for 'LCD'
- iii. List all the customers bearing quotation dated '20-May-2020'
- iii. List all the customers who live in 'U.P.' or 'M. P'.
- iv. Display customers of amt\_quoted as Rs. 15,000.

## Query 10:

- a. Write SQL Statements the update the following table STUDENT with new records.  
STUDENTS

| Student – ID | Student – name | Class | DOB       |
|--------------|----------------|-------|-----------|
| 01           | M. ARSLAN      | MECOM | 12-Apr-82 |
| 02           | JAVARIA AKRAMI | MIT   | 18-Sep-81 |
| 03           | FALAK SHER     | BIT   | 15-Mar-83 |
| 04           | FATIMA         | MECOM | 10-Oct-82 |

- b. Write a SQL Statement, which will display the all students who born in year 1981, from the table STUDENT assuming that STUDENT contains records about 1500 students.

\*\*\*\*\*

## ER Diagram:1

- (a) An insurance agent sells insurance policies to clients. Policies can be of different types such as Vehicle Insurance, Life Insurance, Accident Insurance etc. The agent collects monthly premiums on the policies in the form of cheques of local bank. Database should provide the various details to the user.
  - (i) Identify all entities.
  - (ii) Identify all relations.
  - (iii) E-R Diagram.



## ER Diagram:2

- a. Draw an E-R Model according to following Case study with assumptions. You may make your own assumptions if needed. After identifying Entities, some attributes are given, other you can assume according to your assumptions.
  - I. A company “Ahsan Corporation” deals in selling products to its customers after purchasing products from different vendors, vendors have attributes like vendor id, vendor name, address, phone number etc.
  - II. Company sends purchase order to a vendor and provides the list of items to purchase by the company.
  - III. An item may be purchased from different vendors.
  - IV. Item are stored in the stock after they are received from vendor.
  - V. Customers send Ahsan Corporation, sales order with the list of products to buy from Ahsan Corporation.
  - VI. Products are sent to Customers with the invoice after receiving sales order from customers. Invoice has attributes of invoice number, data etc.
  - VII. Customers make the payments according to invoice after receiving the invoice and make payments on some data.

## The EXISTS function in MySQL:

- EXISTS is a Boolean functions that return TRUE or FALSE; They can be used in a WHERE clause condition. The EXISTS function in SQL is used to check whether the ***result of a nested query is empty (contains no tuples) or not.***
- The result of EXISTS is a Boolean value **TRUE** if the nested query result contains at least one tuple, or **FALSE** if the nested query result contains no tuples.
- example: we have two tables: *employees* and *departments*, and we want to find all employees who are working in each department that has more than 5 employees.

## The EXISTS function in MySQL:

```
SELECT employee_name  
FROM employees e  
WHERE EXISTS (  
    SELECT *  
    FROM employees emp  
    WHERE emp.department_id = e.department_id  
    GROUP BY emp.department_id  
    HAVING COUNT(*) > 5  
);
```

## The EXISTS function in MySQL:

we formulate Query 16 in an alternative form that uses EXISTS as in Q16B:

Query 16. Retrieve the name of each employee who has a dependent with the same first name and is the same sex as the employee.

**Q16:** SELECT E. Fname, E. Lname FROM EMPLOYEE AS E  
WHERE E.Ssn IN ( SELECT D. Essn FROM DEPENDENT AS D  
WHERE E.Fname = D.Dependent\_name AND E.Sex = D.Sex );

**Q16A:** SELECT E.Fname, E.Lname FROM EMPLOYEE AS E, DEPENDENT AS D  
WHERE E.Ssn = D.Essn AND E.Sex = D.Sex  
AND E.Fname = D.Dependent\_name;

## The EXISTS function in MySQL:

**Q16B:** SELECT E.Fname, E.Lname FROM EMPLOYEE AS E  
WHERE EXISTS ( SELECT \* FROM DEPENDENT AS D  
WHERE E.Ssn = D.Essn AND E.Sex = D.Sex AND E.Fname = D. Dependent\_name);

## The NOT EXISTS function in MySQL:

NOT EXISTS(Q) returns TRUE if there are *no tuples in the result of nested query Q*, and returns FALSE otherwise.

Query 6. Retrieve the names of employees who have no dependents.

**Q6:** SELECT Fname, Lname FROM EMPLOYEE  
WHERE NOT EXISTS ( SELECT \* FROM DEPENDENT  
WHERE Ssn = Essn );

In q6, for each EMPLOYEE tuple, the correlated nested query selects all DEPENDENT tuples whose Essn value matches the EMPLOYEE Ssn; if the result is **empty**, no dependents are related to the employee, so we select that **EMPLOYEE** tuple and retrieve its **Fname** and **Lname**.

## The NOT EXISTS function in MySQL:

**Query 3:** List the names of employees who work on *all the projects controlled* by department number 5.

Rewrite the Query 3 using EXISTS and NOT EXISTS in MySQL system.

# Different types of join in MySQL:

1. INNER JOIN
2. LEFT JOIN (or LEFT OUTER JOIN)
3. RIGHT JOIN (or RIGHT OUTER JOIN)

## INNER JOIN:

- The INNER JOIN returns only the **rows** where there is a **match in both tables**. If there is **no match**, the **row will not be** included in the result set.
- We use an INNER JOIN when we are interested in retrieving only the records that have **related data in both tables**.

e.g.:  
`SELECT *  
FROM employees  
INNER JOIN departments  
ON employees.department_id = departments.department_id;`

Here, only employees who have an associated department will be included.



# Different types of join in MySQL: INNER JOIN

e.g.1:

Employees table:

| EmployeeID | Name  | DepartmentID |
|------------|-------|--------------|
| 1          | Alice | 10           |
| 2          | Bob   | 20           |
| 3          | Carol | 30           |

Departments table:

| DepartmentID | DepartmentName |
|--------------|----------------|
| 10           | HR             |
| 20           | IT             |

## Outcome of Query:

| Name  | DepartmentName |
|-------|----------------|
| Alice | HR             |
| Bob   | IT             |

## Query:

```
SELECT Employees.Name, Departments.DepartmentName
FROM Employees
INNER JOIN Departments
ON Employees.DepartmentID = Departments.DepartmentID;
```

# Different types of join in MySQL: INNER JOIN

e.g.2:

1. employees table:

| employee_id | name    | department_id |
|-------------|---------|---------------|
| 1           | Alice   | 101           |
| 2           | Bob     | 102           |
| 3           | Charlie | 103           |
| 4           | David   | NULL          |
| 5           | Eve     | 102           |

2. departments table:

| department_id | department_name |
|---------------|-----------------|
| 101           | HR              |
| 102           | IT              |
| 103           | Marketing       |
| 104           | Finance         |

| employee_id | name    | department_id | department_name |
|-------------|---------|---------------|-----------------|
| 1           | Alice   | 101           | HR              |
| 2           | Bob     | 102           | IT              |
| 3           | Charlie | 103           | Marketing       |
| 5           | Eve     | 102           | IT              |

# Different types of join in MySQL: LEFT OUTER JOIN

A LEFT JOIN returns **all the rows from the left table** and the **matched rows from the right table**. If **no match** is found, **NULL** values are returned for columns from the right table.

e.g.1:

Employees table:

| EmployeeID | Name  | DepartmentID |
|------------|-------|--------------|
| 1          | Alice | 10           |
| 2          | Bob   | 20           |
| 3          | Carol | 30           |

Departments table:

| DepartmentID | DepartmentName |
|--------------|----------------|
| 10           | HR             |
| 20           | IT             |

**Query:**

```
SELECT Employees.Name, Departments.DepartmentName
FROM Employees
LEFT JOIN Departments
ON Employees.DepartmentID = Departments.DepartmentID;
```

**Outcome of Query:**

| Name  | DepartmentName |
|-------|----------------|
| Alice | HR             |
| Bob   | IT             |
| Carol | NULL           |

**Note:** Left table is the table listed first in the query.  
Right table is the table listed second in the query.

# Different types of join in MySQL: LEFT OUTER JOIN

e.g.2:

Let's assume the EMPLOYEE table has the following data:

| Ssn   | Lname   | Super_ssn |
|-------|---------|-----------|
| 12345 | John    | 54321     |
| 54321 | Alice   | NULL      |
| 67890 | Bob     | 12345     |
| 11223 | Charlie | 54321     |

The LEFT OUTER JOIN ensures that every employee is listed, even if they don't have a supervisor. If there's no supervisor (Super\_ssn is NULL), the result shows NULL for the supervisor's name.

## Query Output:

The output of the query would look like this:

| Employee_name | Supervisor_name |
|---------------|-----------------|
| John          | Alice           |
| Alice         | NULL            |
| Bob           | John            |
| Charlie       | Alice           |

**Note:** When we need all rows from the left table, regardless of matching data in the right table, we use LEFT JOIN operation.

# Different types of join in MySQL: LEFT OUTER JOIN

e.g.3:

```
SELECT employees.name, departments.dept_name  
FROM employees  
LEFT JOIN departments  
ON employees.emp_id = departments.emp_id;
```

Result:

| name    | dept_name |
|---------|-----------|
| Alice   | HR        |
| Bob     | IT        |
| Charlie | NULL      |

- employees :

| emp_id | name    |
|--------|---------|
| 1      | Alice   |
| 2      | Bob     |
| 3      | Charlie |

- departments :

| dept_id | emp_id | dept_name |
|---------|--------|-----------|
| 1       | 1      | HR        |
| 2       | 2      | IT        |
| 3       | 4      | Finance   |

The LEFT JOIN returns all employees. Since Charlie has no department, the dept\_name for Charlie is NULL.

# Different types of join in MySQL: RIGHT INNER JOIN

It returns **all the rows** from the **right table** and the **matched rows** from the **left table**. If **no match** is found, **NULL** values are returned for columns from the left table.

e.g.1:

**Employees Table:**

| EmployeeID | Name  | DepartmentID |
|------------|-------|--------------|
| 1          | Alice | 10           |
| 2          | Bob   | 20           |
| 3          | Carol | 30           |
| 4          | Dave  | NULL         |

**Departments Table:**

| DepartmentID | DepartmentName |
|--------------|----------------|
| 10           | HR             |
| 20           | IT             |
| 30           | Finance        |
| 40           | Marketing      |

## Query:

```
SELECT Employees.Name, Departments.DepartmentName
FROM Employees
RIGHT JOIN Departments
ON Employees.DepartmentID = Departments.DepartmentID;
```

## Outcome of Query:

| Name  | DepartmentName |
|-------|----------------|
| Alice | HR             |
| Bob   | IT             |
| Carol | Finance        |
| NULL  | Marketing      |

# Different types of join in MySQL: RIGHT INNER JOIN

e.g.2:

- employees :

| emp_id | name    |
|--------|---------|
| 1      | Alice   |
| 2      | Bob     |
| 3      | Charlie |

- departments :

| dept_id | emp_id | dept_name |
|---------|--------|-----------|
| 1       | 1      | HR        |
| 2       | 2      | IT        |
| 3       | 4      | Finance   |

Result:

| name  | dept_name |
|-------|-----------|
| Alice | HR        |
| Bob   | IT        |
| NULL  | Finance   |

The RIGHT JOIN returns all departments. Since there's no employee with emp\_id = 4, the name for the Finance department is NULL.

```
SELECT employees.name, departments.dept_name  
FROM employees  
RIGHT JOIN departments  
ON employees.emp_id = departments.emp_id;
```

# Views (Virtual Tables) in SQL

- A **view** in SQL terminology is a **single virtual table** that is derived from other tables. These other tables can be base tables or previously defined views.
- A view does not necessarily exist in physical form; it is considered to be a **virtual table**, in contrast to base tables, whose tuples are always physically stored in the database.
- Some views are read-only, meaning they can only be used to view data, while others can be updatable, allowing modifications to the underlying data.

## Syntax to Create a View:

```
CREATE VIEW view_name AS  
SELECT column1, column2, ...  
FROM table_name  
WHERE condition;
```



# Views (Virtual Tables) in SQL

|                                                                                                                              |                                                                                                                                                                                                                                                                                                                                    |
|------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| A table is a database object that physically stores data.                                                                    | A view is a virtual table that is based on the result of a SELECT query. It doesn't store data physically but shows data derived from one or more tables. It is a stored query.                                                                                                                                                    |
| It consists of rows and columns where the actual and permanent data resides.                                                 | Views don't hold data themselves; they <b>retrieve</b> data from underlying tables whenever they are queried.                                                                                                                                                                                                                      |
| we can <b>insert, update, and delete</b> rows directly in a table.                                                           | we <b>cannot directly insert, update, or delete</b> data in a view if the view is based on complex queries.                                                                                                                                                                                                                        |
| <pre>CREATE TABLE Employees (<br/>  EmployeeID INT PRIMARY KEY,<br/>  Name VARCHAR(100),<br/>  DepartmentID INT<br/>);</pre> | <pre>CREATE VIEW HR_Employees AS<br/>  SELECT Name, DepartmentID<br/>  FROM Employees<br/>  WHERE DepartmentID = 10;</pre> <p><i>//The HR_Employees view will show only employees who belong to the department with DepartmentID = 10. However, it doesn't store this data but retrieves it each time the view is queried.</i></p> |
| <pre>SELECT * FROM Employees;</pre>                                                                                          | <pre>SELECT * FROM HR_Employees;</pre>                                                                                                                                                                                                                                                                                             |

# Views (Virtual Tables) in SQL: Why

- Simplification of Complex Queries
- Improve security by hiding sensitive data.
- Centralize logic for easier maintenance and consistency.
- Reuse complex query logic without duplication.
- Avoiding Repetition of Logic in Queries

```
CREATE TABLE Employees (  
  EmployeeID INT PRIMARY KEY,  
  Name VARCHAR(100),  
  DepartmentID INT  
);
```

```
SELECT * FROM Employees;
```

```
CREATE VIEW HR_Employees AS  
  SELECT Name, DepartmentID  
  FROM Employees  
  WHERE DepartmentID = 10;
```

*//The HR\_Employees view will show only employees who belong to the department with DepartmentID = 10. However, it doesn't store this data but retrieves it each time the view is queried.*

```
SELECT * FROM HR_Employees;
```

```
CREATE VIEW total_sales AS  
SELECT product_id, SUM(amount) AS  
total_amount  
FROM sales  
GROUP BY product_id;
```

—————→ **SELECT \* FROM total\_sales;**

# Views (Virtual Tables) in SQL

## Syntax to Create a View

```
CREATE VIEW view_name AS  
SELECT column1, column2, ...  
FROM table_name  
WHERE condition;
```

```
V1:  CREATE VIEW      WORKS_ON1  
      AS SELECT      Fname, Lname, Pname, Hours  
            FROM      EMPLOYEE, PROJECT, WORKS_ON  
            WHERE      Ssn = Essn AND Pno = Pnumber;  
  
V2:  CREATE VIEW      DEPT_INFO(Dept_name, No_of_emps, Total_sal)  
      AS SELECT      Dname, COUNT (*), SUM (Salary)  
            FROM      DEPARTMENT, EMPLOYEE  
            WHERE      Dnumber = Dno  
            GROUP BY  Dname;
```

EMPLOYEE

|       |       |       |            |       |         |     |        |           |     |
|-------|-------|-------|------------|-------|---------|-----|--------|-----------|-----|
| Fname | Minit | Lname | <u>Ssn</u> | Bdate | Address | Sex | Salary | Super_ssn | Dno |
|-------|-------|-------|------------|-------|---------|-----|--------|-----------|-----|

DEPARTMENT

|       |                |         |                |
|-------|----------------|---------|----------------|
| Dname | <u>Dnumber</u> | Mgr_ssn | Mgr_start_date |
|-------|----------------|---------|----------------|

DEPT\_LOCATIONS

|                |                  |
|----------------|------------------|
| <u>Dnumber</u> | <u>Dlocation</u> |
|----------------|------------------|

PROJECT

|              |                |           |      |
|--------------|----------------|-----------|------|
| <u>Pname</u> | <u>Pnumber</u> | Plocation | Dnum |
|--------------|----------------|-----------|------|

WORKS\_ON

|             |            |       |
|-------------|------------|-------|
| <u>Essn</u> | <u>Pno</u> | Hours |
|-------------|------------|-------|

DEPENDENT

|             |                       |     |       |              |
|-------------|-----------------------|-----|-------|--------------|
| <u>Essn</u> | <u>Dependent_name</u> | Sex | Bdate | Relationship |
|-------------|-----------------------|-----|-------|--------------|

**Figure 5.5**

Schema diagram for the COMPANY relational database schema.

# Views (Virtual Tables) in SQL

**WORKS\_ON1**

|       |       |       |       |
|-------|-------|-------|-------|
| Fname | Lname | Pname | Hours |
|-------|-------|-------|-------|

**DEPT\_INFO**

|           |            |           |
|-----------|------------|-----------|
| Dept_name | No_of_emps | Total_sal |
|-----------|------------|-----------|

**Figure 7.2**

Two views specified on the database schema of Figure 5.5.

- In V1, we did not specify any new attribute names for the view WORKS\_ON1. WORKS\_ON1 inherits the names of the view attributes from the defining tables EMPLOYEE, PROJECT, and WORKS\_ON.
- View V2 explicitly specifies new attribute names for the view DEPT\_INFO, using a one-to-one correspondence between the attributes specified in the CREATE VIEW clause and those specified in the SELECT clause of the query that defines the view.

# Views (Virtual Tables) in SQL

We can now **specify SQL queries on a view—or virtual table**—in the same way we specify queries involving base tables. For example, to retrieve the last name and first name of all employees who work on the ‘ProductX’ project, we can utilize the WORKS\_ON1 view and specify the query as in QV1:

**QV1:**

```
SELECT Fname, Lname  
FROM WORKS_ON1  
WHERE Pname = ‘ProductX’;
```

one of the main advantages of a view is to simplify the specification of certain queries. Views are also used as a security and authorization mechanism.

A view is supposed to be always up-to-date; if we modify the tuples in the base tables on which the view is defined, the view must automatically reflect these changes.

# Views (Virtual Tables) in SQL

If we do not need a view anymore, we can use the DROP VIEW command to dispose of it.

For example, to get rid of the view V1, we can use the SQL statement in V1A:

**V1A:**

```
DROP VIEW WORKS_ON1;
```

# Views (Virtual Tables) in SQL

- A view with a single defining table is updatable if the view attributes contain the primary key of the base relation, as well as all attributes with the NOT NULL constraint that do not have default values specified.
- Views defined on multiple tables using joins are generally not updatable.
- Views defined using grouping and aggregate functions are not updatable.

```
UV1:  UPDATE WORKS_ON1  
      SET      Pname = 'ProductY'  
      WHERE    Lname = 'Smith' AND Fname = 'John'  
            AND Pname = 'ProductX';
```



This query can be mapped into several updates on the base relations to give the desired update effect on the view. In addition, some of these updates will create additional side effects that affect the result of other queries.

# The DROP Command

- The DROP command can be used to drop named schema elements, such as tables, domains, types, or constraints.
- It permanently removes the object from the database, and all data associated with that object is lost.
- **1. Dropping a Database:** DROP DATABASE database\_name;
  - `DROP DATABASE university_db; //` permanently deletes the university\_db database and all tables, views, and data within it.
- **2. Dropping a Table:** DROP TABLE table\_name;
  - `DROP TABLE employees; //` permanently removes the employees table and all the data in it.
  - `DROP TABLE table1, table2, table3;`



# The DROP Command

3. Dropping a Column (in a Table): ALTER TABLE table\_name DROP COLUMN column\_name;

- `ALTER TABLE employees DROP COLUMN age;` // drops the column age from the employees table

4. Dropping a View: DROP VIEW view\_name;

- `DROP VIEW employee_view;` // drops a view named employee\_view

5. Dropping a Foreign Key Constraint: ALTER TABLE table\_name DROP FOREIGN KEY constraint\_name;

- `ALTER TABLE employees DROP FOREIGN KEY fk_department_id;` //drops a foreign key constraint named fk\_department\_id on the employees table:

# The DROP Command

## DROP TABLE and Dependent Foreign Keys:

- If we drop a table that is referenced by foreign keys, MySQL will not allow to drop the table if there are any foreign key constraints pointing to it unless we first drop the dependent foreign keys or use cascading actions.
- However, in MySQL, we cannot use the **CASCADE** option directly with the **DROP TABLE** command.
- So, in MySQL, to achieve this (removing a table along with its dependent foreign key constraints), we would need to:
  - Drop Foreign Key Constraints (if any exist).
  - Drop the Table.

Note: Databases like PostgreSQL, which supports **DROP TABLE CASCADE** to automatically remove dependent objects.

**Table 7.2** Summary of SQL Syntax

|                                                                                                                                                                                                                                                                                                                                |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CREATE TABLE <table name> ( <column name> <column type> [ <attribute constraint> ]<br>{ , <column name> <column type> [ <attribute constraint> ] }<br>[ <table constraint> { , <table constraint> } ] )                                                                                                                        |
| DROP TABLE <table name>                                                                                                                                                                                                                                                                                                        |
| ALTER TABLE <table name> ADD <column name> <column type>                                                                                                                                                                                                                                                                       |
| SELECT [ DISTINCT ] <attribute list><br>FROM ( <table name> { <alias> }   <joined table> ) { , ( <table name> { <alias> }   <joined table> ) }<br>[ WHERE <condition> ]<br>[ GROUP BY <grouping attributes> [ HAVING <group selection condition> ] ]<br>[ ORDER BY <column name> [ <order> ] { , <column name> [ <order> ] } ] |
| <attribute list> ::= ( *   ( <column name>   <function> ( ( [ DISTINCT ] <column name>   * ) ) )<br>{ , ( <column name>   <function> ( ( [ DISTINCT ] <column name>   * ) ) ) } )                                                                                                                                              |
| <grouping attributes> ::= <column name> { , <column name> }                                                                                                                                                                                                                                                                    |
| <order> ::= ( ASC   DESC )                                                                                                                                                                                                                                                                                                     |
| INSERT INTO <table name> [ ( <column name> { , <column name> } ) ]<br>( VALUES ( <constant value> , { <constant value> } ) { , ( <constant value> { , <constant value> } ) }<br>  <select statement> )                                                                                                                         |
| DELETE FROM <table name><br>[ WHERE <selection condition> ]                                                                                                                                                                                                                                                                    |
| UPDATE <table name><br>SET <column name> = <value expression> { , <column name> = <value expression> }<br>[ WHERE <selection condition> ]                                                                                                                                                                                      |
| CREATE [ UNIQUE ] INDEX <index name><br>ON <table name> ( <column name> [ <order> ] { , <column name> [ <order> ] } )<br>[ CLUSTER ]                                                                                                                                                                                           |
| DROP INDEX <index name>                                                                                                                                                                                                                                                                                                        |
| CREATE VIEW <view name> [ ( <column name> { , <column name> } ) ]<br>AS <select statement>                                                                                                                                                                                                                                     |
| DROP VIEW <view name>                                                                                                                                                                                                                                                                                                          |

NOTE: The commands for creating and dropping indexes are not part of standard SQL.

***SQL TRIGGERS WILL BE DISCUSSED LATER....***

**THANK YOU**